

Curso: CC3086 Programación de Microprocesadores

Ciclo 1 de 2026

Daniel Rivas

Tomas Cleverson

Laboratorio 05 - SysTick



Secuencia asignada: Secuencia 7 (Período base: 1.2 s)

Placa: STM32F446RE (Nucleo-64)

Fecha: 3/25/2026

PARTE 01 - Diseño

En esta parte se presenta el diseño de la solución antes de programar, incluyendo definición de pines, registros a utilizar, diagrama del circuito, diagrama de flujo, lógica del botón y una explicación general del funcionamiento.

a) Definición de pines GPIO (entrada y salida)

Tabla 1

Definiciones de pines

Etiqueta placa	Puerto MCU	Uso	Modo	
D2	PA10	LED 1 (extremo izquierdo)	Salida (push-pull)	
D3	PB3	LED 2	Salida (push-pull)	
D4	PB5	LED 3 (centro)	Salida (push-pull)	
D5	PB4	LED 4	Salida (push-pull)	
D6	PB10	LED 5 (extremo derecho)	Salida (push-pull)	▼
USER Button	PC13	Cambio de velocidad (2x)	Entrada (pull-up en placa)	

Nota. Los pines corresponden a la placa STM32F446RE Nucleo-64. Los LEDs se configuran como salidas digitales en modo push-pull y el botón PC13 se utiliza como entrada con resistencia de pull-up interna.

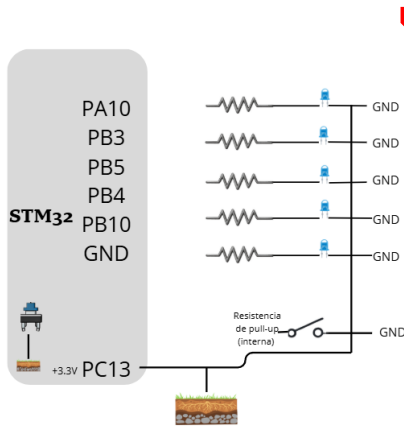
b) Registros para configuración y control ▼

- RCC->AHB1ENR: habilitar reloj de GPIOA, GPIOB y GPIOC.
- GPIOA->MODER y GPIOB->MODER: configurar PA10, PB3, PB4, PB5 y PB10 como salidas.

- GPIOC->MODER: configurar PC13 como entrada.
- GPIOA->ODR y GPIOB->ODR: controlar el encendido/apagado de los LEDs.
- GPIOC->IDR: lectura del estado del Blue Button (PC13).
- SysTick (SysTick_Config / SysTick->LOAD / SysTick->CTRL): temporización para base de 1 ms.

c) Diagrama del circuito

Figura 1



Nota. El diagrama muestra la conexión de los LEDs a los pines del STM32 con resistencias y retorno a GND, así como el uso del botón PC13 con resistencia de pull-up interna.

d) Diagrama de flujo del comportamiento del programa

El diagrama de flujo este en el PDF subido respectivamente, no lo puse como imagen porque no se podía leer con claridad

e) Lógica de control del Blue Button (Normal y 2x)

PC13 trabaja con pull-up, por lo que al presionar el botón el pin se lee como 0. El programa detecta el flanco de bajada, aplica un pequeño retardo de debouncing y confirma que el botón siga presionado. Luego invierte la bandera fast_mode y espera a que el botón se libere para no contar múltiples cambios por una sola pulsación. El cambio de velocidad se implementa modificando el umbral de tiempo: 1200 ms (normal) y 600 ms (2x).

f) Explicación breve del funcionamiento del sistema

El sistema utiliza SysTick como base de tiempo de 1 ms. En cada interrupción se incrementa un contador global. Cuando el contador alcanza el valor correspondiente al período (1200 ms o 600 ms), el programa actualiza los pines de salida según el patrón de la Secuencia 7 y avanza al siguiente estado. El botón de usuario alterna entre velocidad normal y 2x.

PARTE 02 - Implementación

En esta parte se describe la implementación del sistema en la placa STM32, la construcción del circuito y las pruebas realizadas para verificar el funcionamiento correcto de la secuencia y el control de velocidad mediante el Blue Button.

- a) Verificación de que la secuencia de LEDs se ejecute correctamente según el temario.
- b) Verificación de que el período base observado corresponda al indicado (1.2 s).
- c) Verificación de que el Blue Button permita duplicar la velocidad (2×).
- d) Verificación de que al presionar nuevamente el botón se regrese a velocidad normal.

Respuestas a las preguntas

1) Al ejecutar la secuencia de LEDs, ¿qué cambios debieron realizar en el programa para lograr que el período observado coincidiera con el período base indicado en el temario? Explique qué factores influyen en la precisión del tiempo generado con SysTick.

Respuesta:

Se configuró el SysTick para que genere un conteo de tiempo en milisegundos (cada 1 ms). Luego se utilizó un contador que va sumando esos milisegundos, y cuando llega a 1200 ms (1.2 segundos), se cambia el estado de los LEDs. De esta forma se logró que el tiempo observado coincidiera con el período base del ejercicio.

La precisión del tiempo depende principalmente de que el reloj del microcontrolador esté bien configurado, de cómo se ajusta el temporizador (SysTick) y de que el programa no tenga muchas interrupciones o tareas que retrasen la ejecución.

2) Al duplicar la velocidad utilizando el Blue Button, ¿qué parte del programa tuvo que modificarse o controlarse para que la secuencia cambiara correctamente de velocidad? Explique cómo su programa gestiona el cambio entre velocidad normal y velocidad 2×.

Respuesta:

Se controla una bandera global `fast_mode` que cambia con cada pulsación válida del botón. SysTick permanece a 1 ms; lo que se ajusta es el umbral del contador: 1200 ms en modo normal y 600 ms en modo 2×. El programa detecta la pulsación (flanco de bajada), aplica debouncing, alterna `fast_mode` y espera la liberación del botón para evitar múltiples toggles por rebote.

3) Si el programa utilizara un retardo basado en ciclos de CPU (por ejemplo, un bucle vacío) en lugar de SysTick, ¿qué ventajas y desventajas tendría respecto a la solución implementada en el laboratorio?

Respuesta:

Ventaja: es más simple de escribir porque no requiere configurar SysTick ni interrupciones. Desventajas: es un retardo bloqueante, dificulta atender el botón u otras tareas, y su tiempo depende del reloj de CPU y de las optimizaciones del compilador. En general es menos preciso y menos escalable que una base de tiempo con SysTick.

4) Explique qué ocurriría si se agregaran más LEDs al sistema utilizando el mismo puerto GPIO. ¿Qué aspectos del diseño del programa deberían modificarse para soportar esta ampliación?

Respuesta:

Se deben habilitar y configurar como salida los pines adicionales en MODER y extender las máscaras/patrones usados para escribir en los pines. También debe actualizarse el

mapeo entre bits del patrón y pines físicos. La temporización con SysTick puede mantenerse igual; el cambio principal está en la definición de pines, configuración de GPIO y patrones de la secuencia.

5) Durante la implementación del circuito, ¿qué errores de conexión o configuración de GPIO podrían provocar que los LEDs no se comporten como se espera? Explique cómo podría identificar y corregir estos problemas.

Respuesta:

Conexión: LED invertido, falta de GND común, ausencia de resistencias, cable en pin incorrecto o suelto. Configuración: no habilitar el reloj del puerto en RCC, configurar el pin en modo incorrecto, o escribir al registro/puerto equivocado. Para diagnosticar: revisar el cableado, medir voltajes con multímetro, probar encender LEDs con un patrón fijo, y depurar observando cambios en ODR/IDR.

Video:

<https://drive.google.com/file/d/1e9W6RSbXIwuFxmQFu3oZCD0JBR7iOt7D/view?usp=sharing>